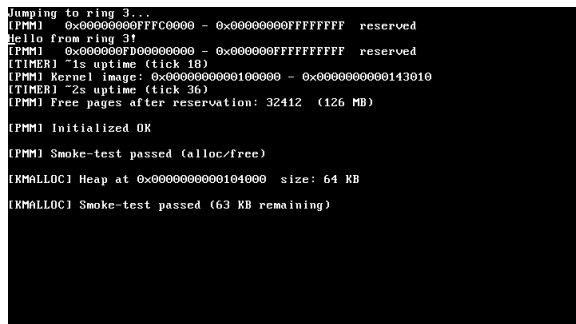


Aarav Daga

Firmware Intern application · aaravdaga@gmail.com · adaga460.github.io

1. A cool thing I've built

I think the coolest thing I've made was a monolithic kernel with a (barely working) userspace. I built it in my sophomore year after getting inspired by my systems programming class (CMPSC 311 at Penn State; the kernel itself was a side project, not an assignment). I got a little ahead of myself with research and side topics, and thought it would be an easy project to complete. It was not, but it was super rewarding and gave me a much deeper understanding of how systems actually work than I expected to have as a sophomore.



```
Jumping to ring 3...
(PMM) 0x00000000FFFC0000 - 0x00000000FFFFFF reserved
Hello from ring 3!
(PMM) 0x000000FD00000000 - 0x00000000FFFFFF reserved
(TIMER) 1s uptime (tick 18)
(PMM) Kernel image: 0x0000000001000000 - 0x000000000143010
(TIMER) 2s uptime (tick 36)
(PMM) Free pages after reservation: 32412 (126 MB)

(PMM) Initialized OK
(PMM) Smoke-test passed (alloc/free)

(KMALLOC) Heap at 0x000000000104000 size: 64 KB
(KMALLOC) Smoke-test passed (63 KB remaining)
```

The kernel booting in QEMU: physical memory manager and heap come up, the timer ticks, and it drops to ring 3 ("Hello from ring 3!"). Source: github.com/ADaga460/microkernel

2. My setup

I use either Vim or VS Code, depending on how much I hate myself that day (feel free to guess which I use when I hate myself). I love Vim because it makes me feel closer to what I'm working with somehow, even though VS Code is much sleeker and more polished. I mainly got into Vim after exclusively writing low-level code.

3. A PCB that resets randomly every few hours

First I'd find out why it reset before touching the hardware. Most chips keep a reset-reason flag (an ESP32 will tell you brownout vs. watchdog vs. crash), so I'd log that plus uptime and free heap on every boot and let it run. If it says brownout, I'd hook up a digital oscilloscope to the supply rail to catch a falling edge, with the trigger set just below the normal value. If it says watchdog or crash, I'd stop staring at the power and go looking for a stuck task or a slow leak instead, since "every few hours" sounds like something filling up. I can confidently say I never HAD to chase a random reset like this, but the crash in question 5 was the same idea: a queue filling up until the board fell over.

4. What firmware/embedded engineers do wrong

Dynamic memory allocation in fact sucks and should be avoided whenever possible. Unless you're a senior senior senior guy who really knows what they're doing, I wouldn't trust it. Secondly, not touching grass and smelling the flowers. Truly a lost art these days.

5. A sketchy fix I'm secretly proud of

I think the sketchiest firmware fix I ever did was implementing what I expertly called the "Tactical Amnesia" protocol on an ESP32 right before a live demo. We were building a real-time audio streamer and it straight-up kept crashing because the microphone queue was filling up faster than the Bluetooth path could clear it (a brand new issue which we were pissed to discover). I didn't have time to fix the root concurrency issue, so I wrote a janky macro that just silently threw incoming audio frames in the trash whenever the queue had less than 10% free. It turned a catastrophic system crash into an imperceptible 20 ms micro-stutter, and I'm secretly super proud of it because it completely saved the 30-minute presentation.